

BMS Predictive Maintenance

ML Pipeline Report

SOH Estimation Using Classical Machine Learning
Battery A (5s1p, LG 18650HG2 NMC)

6 ML Models x 2 Feature Strategies = 12 Pipelines
Random Forest | Extra Trees | XGBoost | Gradient Boosting | LightGBM | CatBoost

Birol - BMS Predictive Maintenance Project

Table of Contents

1.	Project Overview	3
2.	Data Description	4
3.	Pipeline Steps (Step 1-9)	5
4.	ML Models Explained	12
5.	Results & Analysis	16
6.	Key Findings	18
7.	Limitations & Caveats	20
8.	Project Structure	21
9.	Next Steps	22

1. Project Overview

Problem Statement

We have a 5s1p Li-ion battery pack (5 series LG 18650HG2 NMC cells) modeled in MATLAB/Simulink with Simscape Battery. The battery is repeatedly charged and discharged using the UDDS (Urban Dynamometer Driving Schedule) drive cycle, simulating real-world urban driving conditions.

Over 100 charge/discharge cycles, the battery degrades: SOH (State of Health) drops from 100% to approximately 80%, internal resistance increases, capacity decreases, and cell-to-cell imbalance grows. SOH = 80% is defined as End of Life (EOL).

Objective

Build Machine Learning models that predict SOH from the battery's operational data (voltage, current, temperature, SOC) - signals that a real Battery Management System (BMS) can observe. The model must NOT use any information that wouldn't be available in a real deployment (e.g., cycle number, direct capacity measurement).

Why Does This Matter?

In real electric vehicles, you cannot open the battery pack to directly measure capacity. The BMS must estimate SOH from sensor data alone. Accurate SOH estimation enables: predictive maintenance (replace before failure), accurate range estimation, warranty decisions, and battery second-life evaluation.

Approach

We evaluate 6 classical ML models, each with two feature selection strategies (all features vs. top 30), resulting in 12 pipeline variations. Each variation follows the same 9-step pipeline, differing only in the model and feature count at Steps 7-9.

Model	Type	Library
Random Forest	Bagging	sklearn
Extra Trees	Bagging	sklearn
XGBoost	Boosting	xgboost
Gradient Boosting	Boosting	sklearn
LightGBM	Boosting	lightgbm
CatBoost	Boosting	catboost

2. Data Description

Property	Value
Source	Simulink Recording (.mat, HDF5 format)
Battery	5s1p LG 18650HG2 NMC
Drive Cycle	UDDS (Urban Dynamometer Driving Schedule)
Sampling Rate	1 Hz (1 sample per second)
Total Duration	~640 hours (~100 cycles)
Total Rows	~2.3 million
SOH Range	100% to 80%

Available Signals (8 + 1)

Signal	Unit	Description
SOC	%	State of Charge (0-100%)
Current	A	Pack current (charge/discharge)
Cell_V_Max	mV->V	Highest voltage cell among 5
Cell_V_Min	mV->V	Lowest voltage cell among 5
Cell_V_Avg	mV->V	Average voltage of 5 cells
Cell_T_Max	C	Hottest cell temperature
Cell_T_Min	C	Coldest cell temperature
Cell_T_Avg	C	Average temperature of 5 cells
SOH	%	Ground truth (Simscape Fade formula)

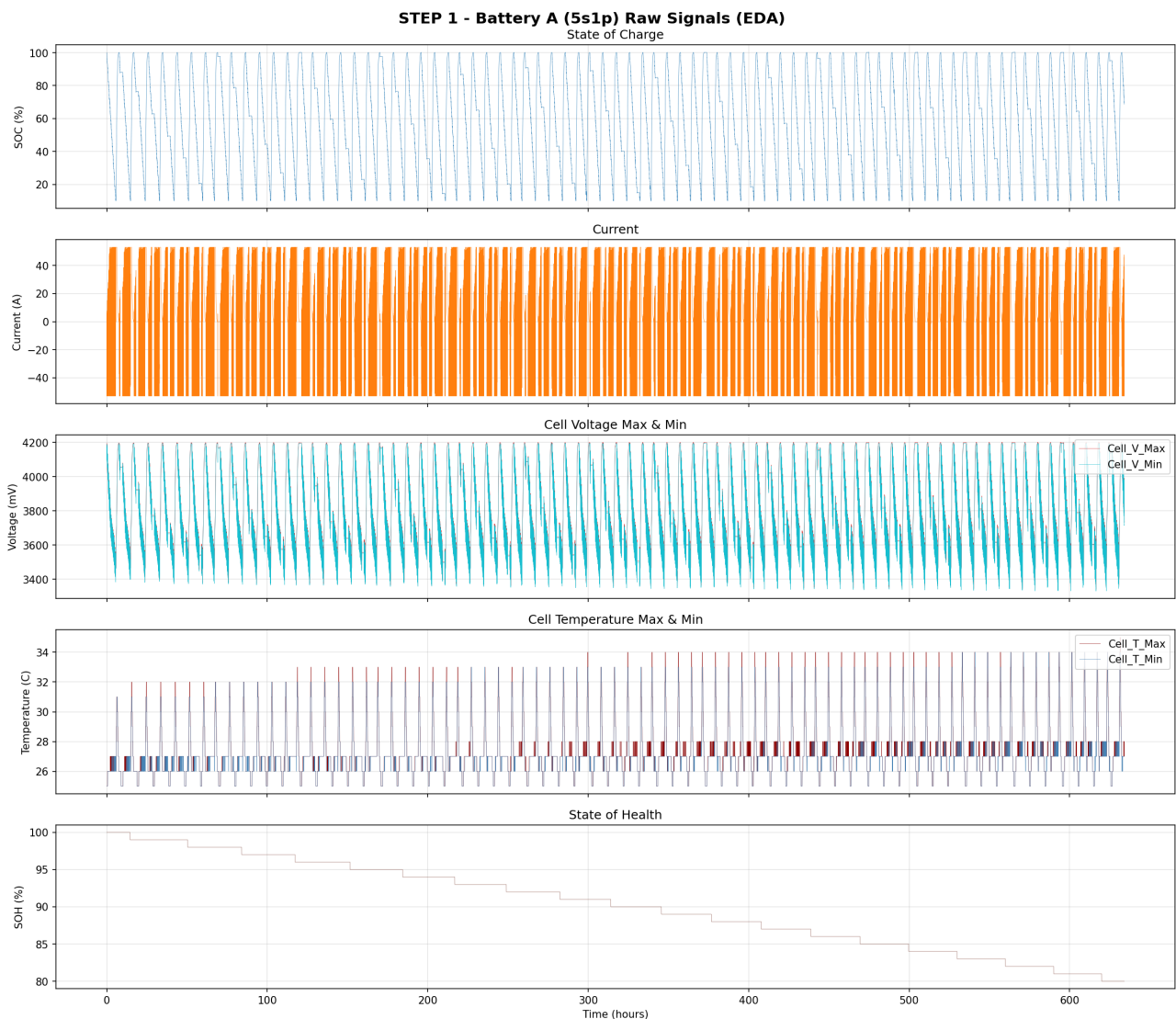
Note: SOH is used only as the target variable (label) for training. It is computed from the Simscape Fade formula and represents the ground truth. In real-world deployment, SOH would not be available - the model must predict it.

3. Pipeline Steps

All 12 pipeline variations follow the same 9-step process. Steps 1-6 are identical across all models. Steps 7-9 differ in feature selection strategy and model choice.

Step 1: Raw Signal Visualization (EDA)

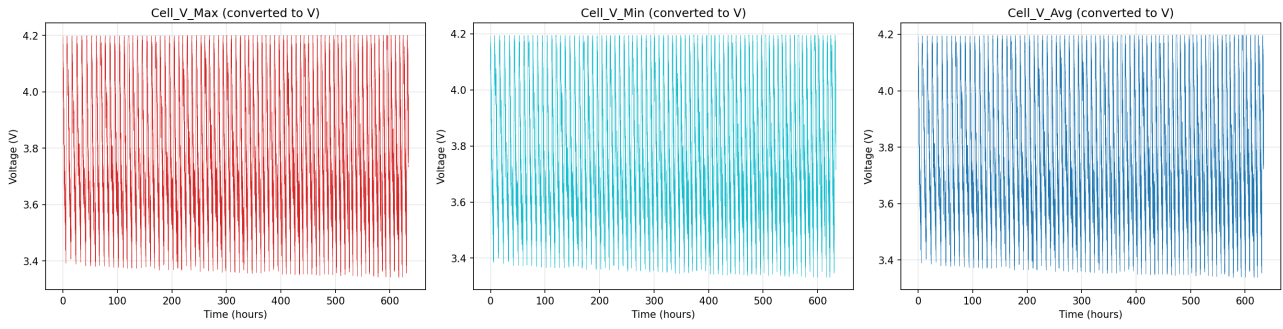
The .mat file is read using h5py (HDF5 format). 9 signal streams are extracted into a pandas DataFrame where each row represents 1 second of data. Raw signals are visualized in 5 subplots: SOC, Current, Voltage (Max & Min), Temperature (Max & Min), and SOH.



Step 2: mV to V Conversion

Simscape outputs voltage signals in millivolts. Cell_V_Max, Cell_V_Min, and Cell_V_Avg are divided by 1000 to convert to Volts. This is a preprocessing step - no information is lost.

STEP 2 - Voltage Signals After mV -> V Conversion



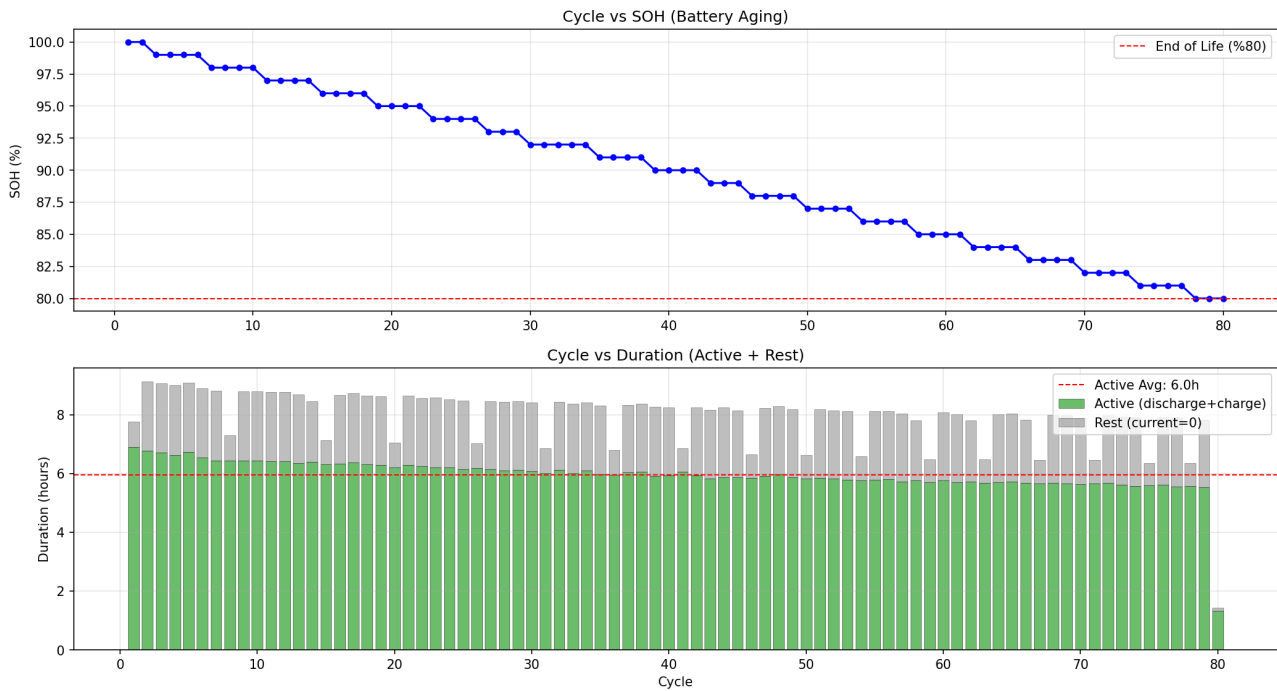
Step 3: Cycle Boundary Detection

One cycle consists of a full discharge (SOC 100% -> ~10%) followed by a full charge (SOC ~10% -> 100%). Cycle boundaries are detected by finding peak points in the SOC signal (SOC > 99%). A minimum distance filter (>5000 seconds between peaks) prevents false detections. Peak-to-peak defines one complete cycle.

Key definitions:

- Active period: $|Current| > 0.5A$ (battery is charging or discharging)
- Rest period: $|Current| \leq 0.5A$ (battery is idle)

STEP 3 - Cycle Analysis (SOH & Duration)



Step 4: Cycle-Level Feature Extraction

This is the most critical step. We compress 2.3 million rows of 1 Hz raw data into ~100 rows (1 row per cycle). Each row contains ~85 features computed from that cycle's raw data. Three categories of features are extracted:

A) Raw Signal Statistics (56 features)

For each of 8 signals, 7 statistics are computed: min, max, mean, std, range, discharge_end (value at SOC minimum), and charge_start (value just after SOC minimum). This gives $8 \times 7 = 56$ features.

B) Domain-Specific Features (9 groups)

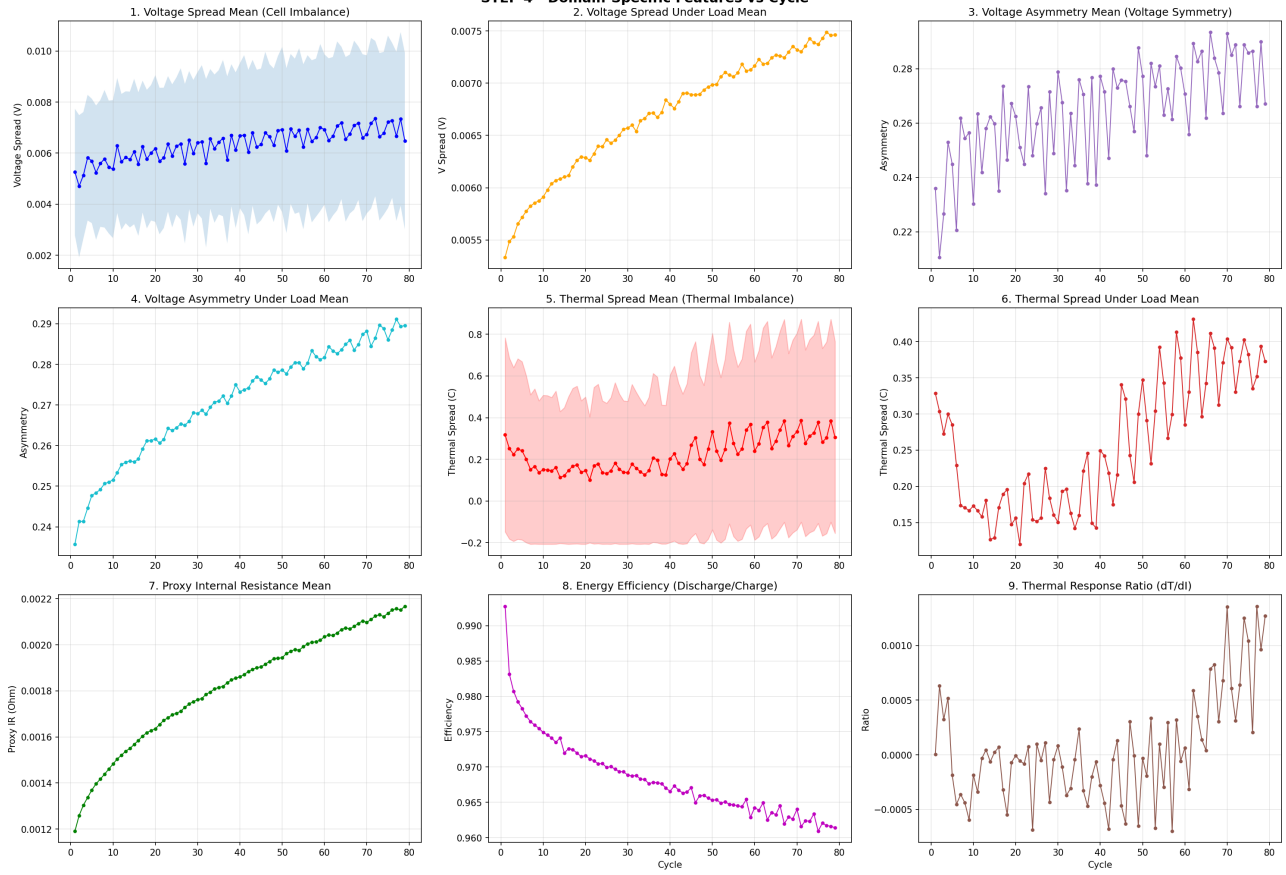
These are battery-aging indicators designed with domain knowledge. Each feature captures a different aspect of battery degradation:

Feature	What It Measures	Aging Trend
Voltage Spread	Cell voltage imbalance	Increases
V Spread Under Load	Imbalance during active periods	Increases
Voltage Asymmetry	Voltage distribution symmetry	Deviates from 1.0
V Asymmetry Under Load	Same, active periods only	Deviates from 1.0
Thermal Spread	Cell temperature difference	Increases
T Spread Under Load	Same, active periods only	Increases
Proxy Internal Resistance	$ dV/dI $ at large current changes	Increases
Energy Efficiency	Discharge/Charge energy ratio	Decreases
Thermal Response Ratio	Temperature sensitivity to current	Increases

C) Timing Features (9 features)

charge_time_to_50/80/95/full, discharge_duration, soc_range, cycle_duration, active_duration, rest_duration. Total: ~85 features per cycle.

STEP 4 - Domain-Specific Features vs Cycle



Step 5: Trend Features

Why Are Trend Features Needed?

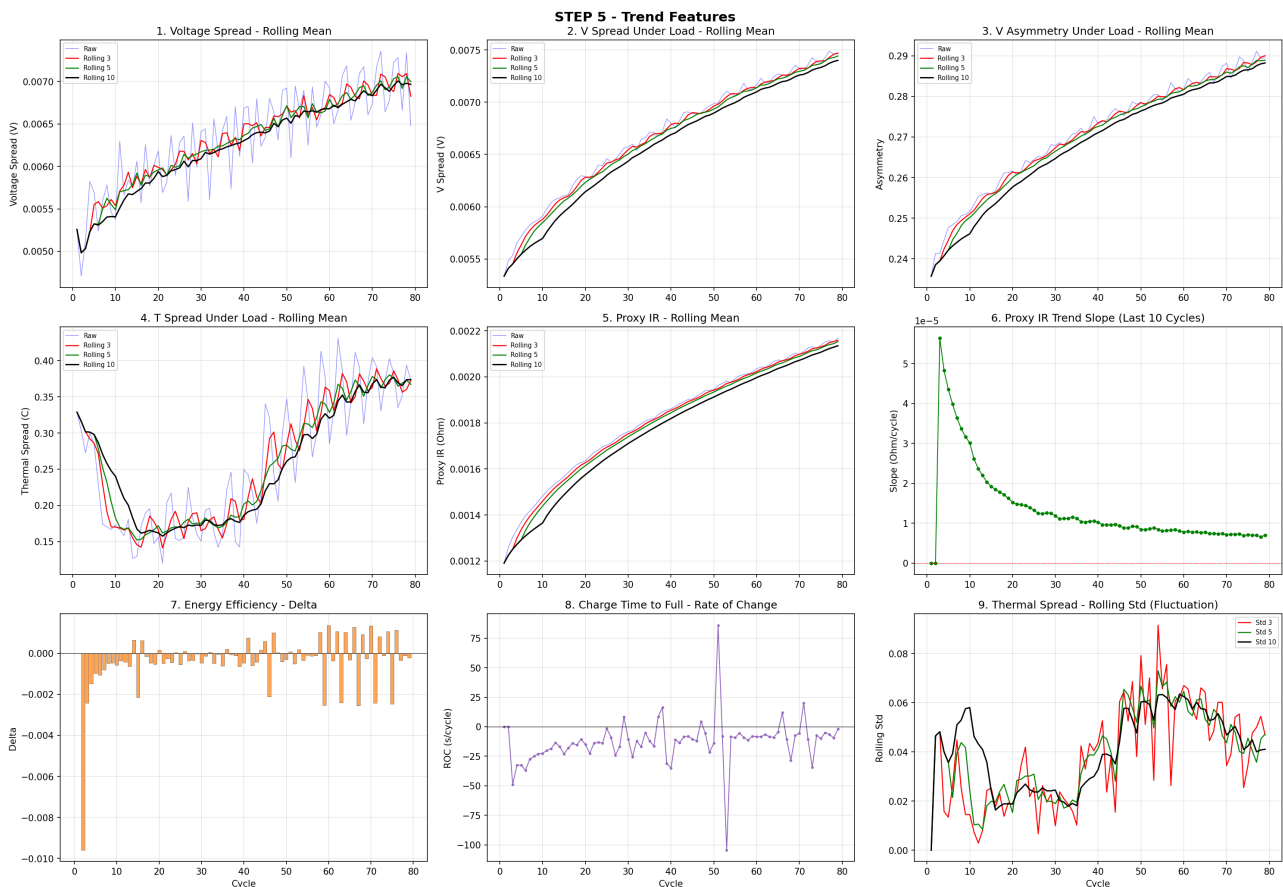
Tree-based ML models treat each row independently - they have no memory of previous cycles. When predicting cycle 50's SOH, the model doesn't know what happened in cycle 49. Two batteries might both have voltage_spread = 0.015V at cycle 50, but one is increasing (getting worse) and the other is decreasing (stabilizing). Without trends, the model gives both the same prediction.

What Trend Features We Compute

For 17 carefully selected domain features, the following are computed:

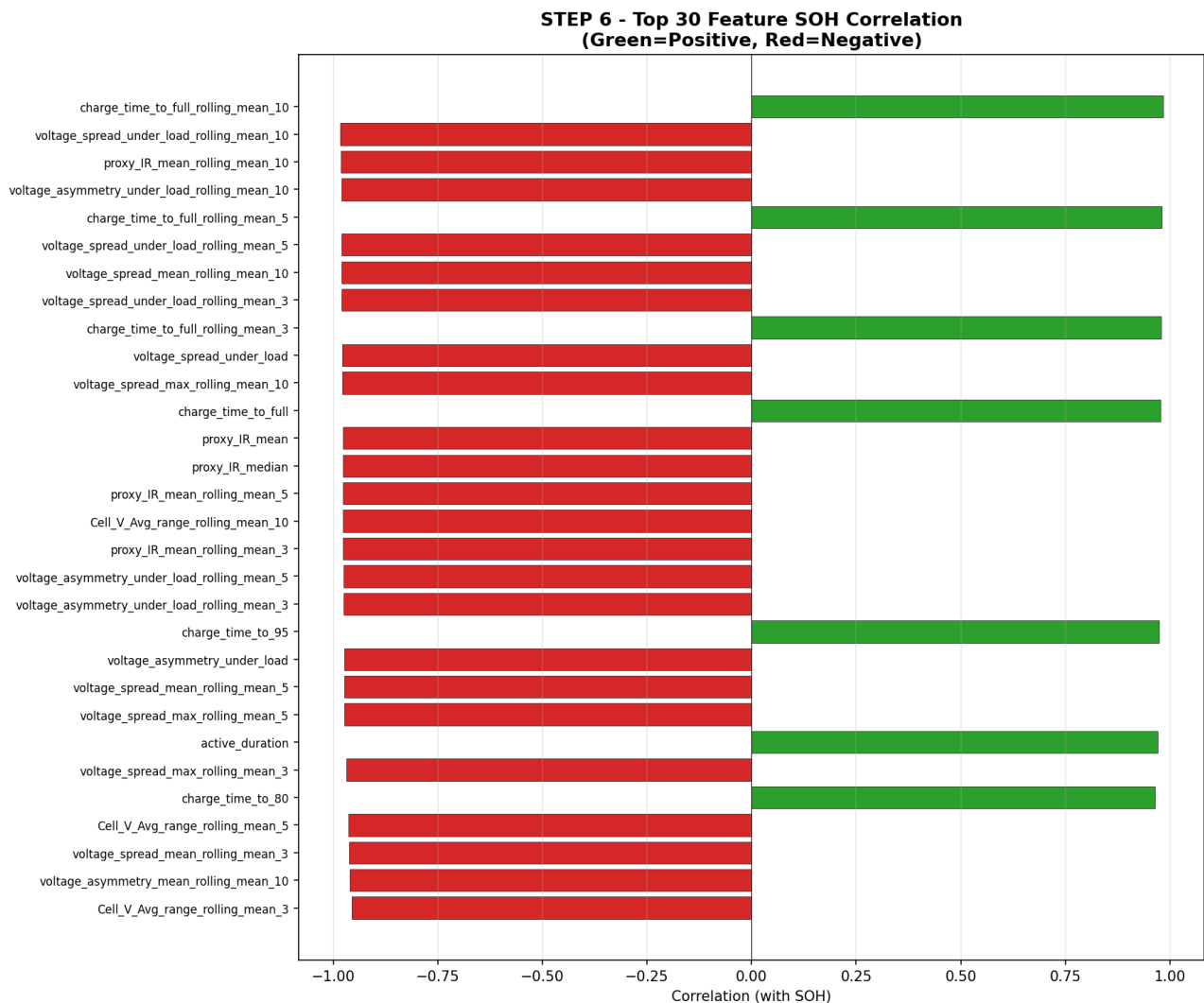
Trend Type	Description	Per Feature
Rolling Mean (3,5,10)	Average of last N cycles	3
Rolling Std (3,5,10)	Volatility over last N cycles	3
Delta	Difference from previous cycle	1
Rate of Change	Average change over 2 cycles	1

Additionally, proxy_IR_trend_slope_10 captures the linear regression slope of internal resistance over the last 10 cycles. Result: ~85 features expand to ~205 features. We only compute trends for 17 features (not all 85) to avoid overfitting with ~100 rows.



Step 6: Correlation Analysis

Pearson correlation between each feature and SOH is computed. This reveals which features are most informative for SOH prediction. Features with $|\text{correlation}| > 0.95$ are very strong predictors. Constant features ($\text{std}=0$) are removed.



Step 7: Feature Selection

Strategy A: All Features (~200)

All features are passed to the model. The model handles feature selection internally. Random Forest assigns importance scores; boosting models select features at each split. Advantage: no information lost. Disadvantage: more noise.

Strategy B: Top 30 Features

The 30 features with highest absolute SOH correlation are selected. All others are discarded. Advantage: less noise, faster, more interpretable. Disadvantage: may lose useful nonlinear features. Important: the 'cycle' column is never fed to any model (data leakage prevention).

Step 8: Model Training

Train/Test Split: Interleaved (75/25)

Every 4th cycle goes to the test set (cycles 4, 8, 12, 16, ...). The remaining 75% is used for training. With single-battery data, a chronological split would only test a narrow SOH range. Interleaved ensures the test set covers the full SOH range.

Caveat: This split is 'easy' for the model since neighboring cycles are very similar. Real-world performance is better estimated by 5-Fold Cross Validation.

Sample Weight

Higher weight is assigned to low-SOH cycles during training. The high-SOH region (100-95%) is more populated, so the model naturally learns it well. Upweighting low-SOH forces the model to focus on the critical degradation zone.

Evaluation Metrics

Metric	Formula	Interpretation
MAE	$\text{mean}(\text{actual} - \text{predicted})$	Avg error in %. Lower = better.
RMSE	$\sqrt{\text{mean}((\text{actual} - \text{pred})^2)}$	Penalizes large errors. $\text{RMSE} \gg \text{MAE}$ = outliers.
R2	$1 - (\text{SSE}/\text{SST})$	Variance explained. 1.0 = perfect.

Overfitting Detection

Train and Test metrics are compared. If the model performs much better on training data than test data, it has memorized rather than learned. Our automatic verdict:

- Diff < 0.5%: OK
- Diff 0.5-1.0%: WARNING
- Diff > 1.0%: OVERFITTING?

5-Fold Cross Validation

The entire dataset is split into 5 folds. Each fold serves as test once. Every data point is tested exactly once. CV MAE is more reliable than a single split.

Key insight: Fold 5 (train: cycles 1-64, test: cycles 65-79) is closest to the real-world scenario where the model must predict the future from past data.

Fold	Test Cycles	Scenario
1	1-16	Extrapolation (initial)
2-4	17-64	Interpolation (middle)
5	65-79	Real-world (future prediction)

Step 9: Hyperparameter Tuning (Grid Search)

Hyperparameters are model settings defined before training (not learned from data). Grid Search systematically tries all combinations of specified parameter values using 5-Fold CV on the train set only. The test set is never used during tuning.

Process:

1. Define parameter grid (e.g., `max_depth: [3, 5, 8, 10]`)
2. For each combination, run 5-Fold CV on the train set
3. Select the combination with lowest CV MAE
4. Retrain on full train set with best parameters
5. Evaluate on held-out test set

Important: With small datasets (~100 samples), Grid Search can find parameters that overfit to CV folds but perform worse on the actual test set. When tuned performs worse than baseline, the baseline should be preferred.

4. ML Models Explained

Random Forest

Type: Bagging (Bootstrap Aggregating)

How it works:

1. Create N independent decision trees, each trained on a random subset of data
2. Each tree considers a random subset of features at each split
3. Final prediction = average of all trees' predictions

The randomness (both in data sampling and feature selection) ensures trees are diverse. Averaging diverse predictions reduces variance and prevents overfitting.

Parameter	Default	What It Controls
n_estimators	300	Number of trees
max_depth	10	Maximum tree depth
min_samples_split	5	Min samples to split a node
min_samples_leaf	2	Min samples per leaf

Extra Trees (Extremely Randomized Trees)

Type: Bagging (even more randomized than RF)

The key difference from Random Forest: instead of finding the optimal split threshold, Extra Trees uses a random threshold. This additional randomness further reduces overfitting and makes it faster (no need to evaluate all possible splits).

Strengths: Lowest variance among tree models, fastest bagging method, excellent on small datasets. Weakness: Slightly higher bias than RF.

XGBoost (Extreme Gradient Boosting)

Type: Boosting (sequential)

How it works:

1. Start with a simple prediction (e.g., mean SOH)
2. Build a tree that predicts the residual error
3. Add this tree's predictions (scaled by learning rate) to the total
4. Repeat: each new tree corrects mistakes of all previous trees

Key advantage: Built-in L1/L2 regularization (reg_alpha, reg_lambda) helps prevent overfitting. Generally high accuracy on medium-to-large datasets.

Parameter	Default	What It Controls
n_estimators	300	Number of boosting rounds
max_depth	6	Tree depth (shallower than RF)
learning_rate	0.05	Contribution of each tree
subsample	0.8	Fraction of data per tree
reg_alpha/lambda	0.1/1.0	L1/L2 regularization

LightGBM (Light Gradient Boosting Machine)

Type: Boosting (leaf-wise growth)

Unlike XGBoost which grows trees level-by-level, LightGBM expands the leaf with highest loss reduction. This is faster and often more accurate on large datasets, but more prone to overfitting on small datasets.

Strengths: Fastest boosting model, excellent on large data.

Weakness: Overfits easily on small data (~100 rows) - confirmed by our results.

Gradient Boosting (sklearn)

Type: Boosting (vanilla implementation)

Same concept as XGBoost but sklearn's native implementation. No extra dependencies needed. Slower than XGBoost/LightGBM but part of the standard sklearn ecosystem. Lacks XGBoost's L1/L2 regularization options.

CatBoost (Categorical Boosting)

Type: Boosting (ordered boosting)

Special feature: Uses 'ordered boosting' which computes residuals using only data points that come before the current one (in a random permutation). This reduces 'prediction shift' - a subtle form of overfitting in standard gradient boosting.

Strengths: Best categorical feature handling, ordered boosting reduces overfitting.

Weakness: Slower than LightGBM, requires catboost package.

Model Comparison Summary

Aspect	Bagging (RF, ET)	Boosting (XGB, LGB, GB, CB)
Training	Parallel (independent)	Sequential (corrective)
Overfitting Risk	Low	Medium-High
Small Data	Robust	Prone to overfit
Feature Scaling	Not needed	Not needed
Extrapolation	Cannot	Cannot
Speed	Medium	Fast (XGB, LGB)

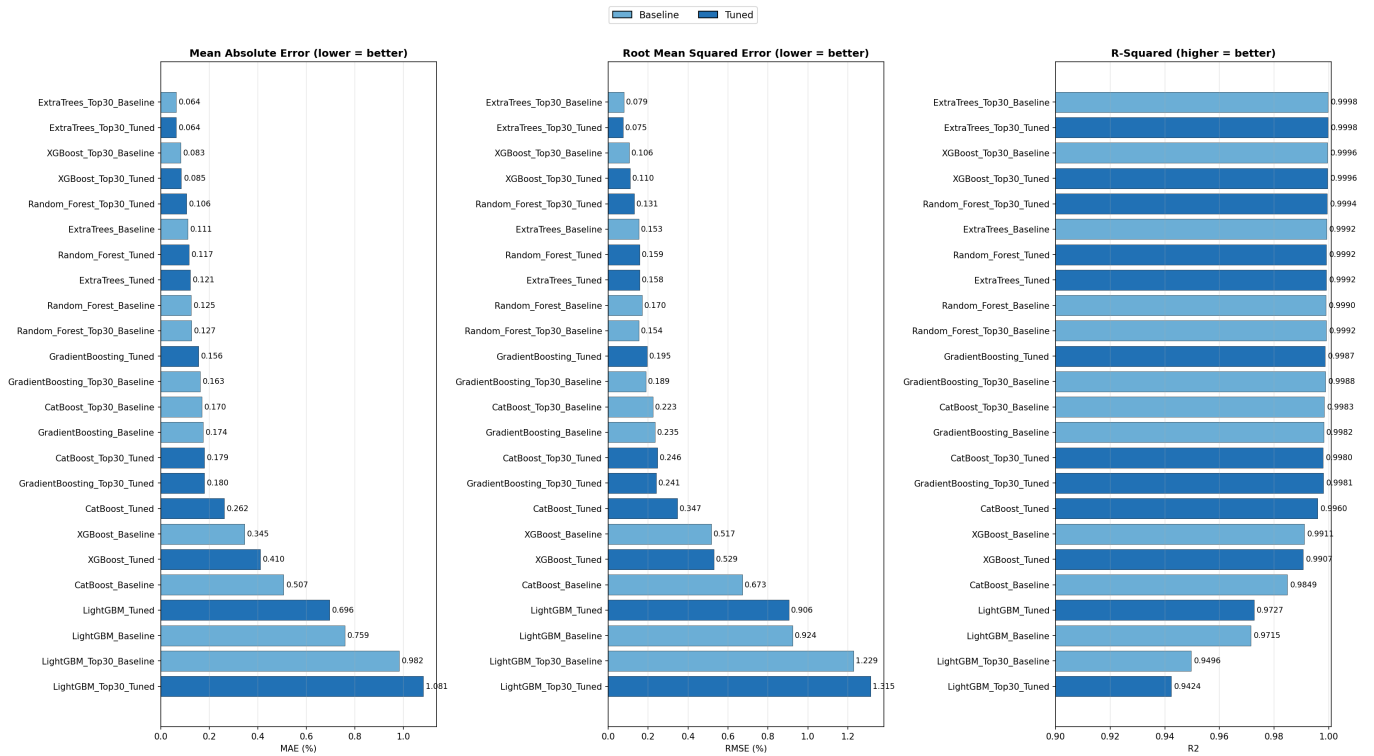
5. Results & Analysis

All Models Ranked by Test MAE

The following table shows all 24 model variations (6 models x 2 feature strategies x baseline/tuned) ranked by test set MAE. All metrics are computed on the 25% held-out test set (interleaved split).

#	Model	MAE	RMSE	R2	OvFit
1	ExtraTrees_Top30_BL	0.064%	0.079%	0.9998	OK
2	ExtraTrees_Top30_TN	0.064%	0.075%	0.9998	OK
3	XGBoost_Top30_BL	0.083%	0.106%	0.9996	OK
4	XGBoost_Top30_TN	0.085%	0.110%	0.9996	OK
5	RF_Top30_TN	0.106%	0.131%	0.9994	OK
6	ExtraTrees_BL	0.111%	0.153%	0.9992	OK
7	RF_TN	0.117%	0.159%	0.9992	OK
8	ExtraTrees_TN	0.121%	0.158%	0.9992	OK
9	RF_BL	0.125%	0.170%	0.9990	OK
10	RF_Top30_BL	0.127%	0.154%	0.9992	OK
11	GradBoost_TN	0.156%	0.195%	0.9987	OK
12	GradBoost_Top30_BL	0.163%	0.189%	0.9988	OK
13	CatBoost_Top30_BL	0.170%	0.223%	0.9983	OK
14	GradBoost_BL	0.174%	0.235%	0.9982	OK
15	CatBoost_Top30_TN	0.179%	0.246%	0.9980	OK
16	GradBoost_Top30_TN	0.180%	0.241%	0.9981	OK
17	CatBoost_TN	0.262%	0.347%	0.9960	OK
18	XGBoost_BL	0.345%	0.517%	0.9911	OK
19	XGBoost_TN	0.410%	0.529%	0.9907	OK
20	CatBoost_BL	0.507%	0.673%	0.9849	OK
21	LightGBM_TN	0.696%	0.906%	0.9727	WARN
22	LightGBM_BL	0.759%	0.924%	0.9715	WARN
23	LightGBM_Top30_BL	0.982%	1.229%	0.9496	OK
24	LightGBM_Top30_TN	1.081%	1.315%	0.9424	WARN

ML Model Comparison - SOH Estimation



6. Key Findings

Finding 1: Best Model - Extra Trees Top30 (MAE = 0.064%)

Extra Trees with Top 30 features achieved the lowest test MAE. The added randomness in split selection reduces overfitting, and 30 well-correlated features provide a clean, noise-free signal for the model.

Finding 2: Top 30 Features Often Beat All Features

Model	All Features MAE	Top 30 MAE	Winner
Extra Trees	0.111%	0.064%	Top 30
XGBoost	0.345%	0.083%	Top 30
Random Forest	0.125%	0.106%	Top 30
Gradient Boosting	0.156%	0.163%	All (marginal)
CatBoost	0.262%	0.170%	Top 30
LightGBM	0.696%	0.982%	All

For most models, removing noise (Top 30) helps more than keeping all information (~200 features). The exception is LightGBM, which performs poorly in both cases.

Finding 3: Bagging > Boosting on Small Data

Bagging models (Random Forest, Extra Trees) consistently outperform boosting models (XGBoost, LightGBM, etc.) on this dataset. With only ~100 training samples, boosting's sequential error correction tends to overfit to noise rather than learn true patterns.

Finding 4: Tuning Doesn't Always Help

In several cases (XGBoost Top30, LightGBM Top30, GradientBoosting Top30), the tuned model performed worse than baseline. With small datasets, Grid Search can find parameters that overfit to CV folds but generalize poorly to the test set. The baseline (hand-tuned) parameters are often safer.

Finding 5: LightGBM Struggles on Small Data

LightGBM consistently showed the worst results (MAE 0.70-1.08%). Its leaf-wise growth strategy is optimized for large datasets. With ~100 rows, it overfits (WARNING flags in multiple overfitting checks). This is a known limitation.

Finding 6: Interleaved Split MAE is Optimistic

The reported MAEs use interleaved split where neighboring cycles are in train/test. CV MAE is typically 10-20x higher because it tests on unseen SOH regions. Real-world performance is closer to CV Fold 5 (chronological extrapolation).

Tree-based models cannot extrapolate - they can only predict values within the range they have seen in training. This is

a fundamental structural limitation.

7. Limitations & Caveats

Single Battery Data

All training and testing used Battery A only. Real-world validation requires testing on Battery B (different production batch, unseen data). This is planned as the next phase (Inference pipeline).

Interleaved Split vs. Real-World

Method	What It Tests	Typical MAE
Interleaved (default)	Interpolation between neighbors	~0.1%
5-Fold CV	Generalization across SOH ranges	~1-3%
Chronological (Fold 5)	Future prediction	~2-3%

The interleaved MAE is impressive but does not reflect real-world deployment where the model must predict forward in time.

Extrapolation Limitation

All tree-based models (RF, XGBoost, LightGBM, etc.) cannot extrapolate. If trained on SOH 100-85%, the model cannot predict SOH 84% - the prediction will plateau at the lowest training value. Deep learning models (LSTM, GRU) can potentially overcome this limitation by learning temporal patterns.

Simulated vs. Real Data

This data comes from Simulink simulation, not real-world battery testing. Real data would have more noise, sensor drift, and environmental variations. Model performance on real data may differ.

8. Project Structure

```
1-ML/
|-- Random_Forest/
|   |-- Random_Forest.py           (All features)
|   |-- Random_Forest_Top30.py     (Top 30 features)
|   |-- Output_Random_Forest/
|   |-- Output_Random_Forest_Top30/
|-- LightGBM/
|   |-- LightGBM.py
|   |-- LightGBM_Top30.py
|-- XGBoost/
|   |-- XGBoost.py
|   |-- XGBoost_Top30.py
|-- GradientBoosting/
|   |-- GradientBoosting.py
|   |-- GradientBoosting_Top30.py
|-- ExtraTrees/
|   |-- ExtraTrees.py
|   |-- ExtraTrees_Top30.py
|-- Catboost/
|   |-- CatBoost.py
|   |-- CatBoost_Top30.py
|-- ML_Comparison.py
|-- ML_Comparison.csv
|-- ML_Comparison.png
|-- README.md
|-- ML_Pipeline_Report.pdf         (This document)
```

Output Files Per Model

File	Content
step[1-9]_.png	Visualization for each pipeline step
df_step[3-9]_.csv	Data outputs (features, correlation, results)
model_*_baseline.joblib	Trained baseline model weights
model_*_tuned.joblib	Trained tuned model weights
feature_columns.txt	Feature list used (for inference)
Result_*.csv	Summary: comparison + overfitting checks

9. Next Steps

Phase 1 Continuation: Battery B Inference

Apply trained models to Battery B (unseen battery, different production batch). Battery B was run in Simulink until SOH reached 88% (ground truth). The inference pipeline applies the same feature extraction (Steps 1-5), aligns features with Battery A's feature list, and predicts SOH using the trained models.

Phase 2: Transfer Learning

Train on Battery A, fine-tune on Battery B. This addresses the extrapolation limitation by allowing the model to adapt to a new battery's characteristics with minimal additional data.

Phase 2: Deep Learning

Implement sequence models (LSTM, GRU, Transformer) that can learn temporal patterns without requiring manual trend features. These models process sequences of cycles and can potentially extrapolate beyond the training SOH range.

Scenario Expansion

- S2: Weak Cell (Cell 60403 with 18.5% higher internal resistance)
- S3: Thermal Problem (Cell 60403, thermal fault injection)
- S4: Progressive R0 Increase (gradual internal resistance growth)